Bilkent University Electrical and Electronics Department

EE102-01 Project Report:

Sign Language Recognition and Display System

Muhammed Mesud Bilirgen – 22301965 –12 May 2025

Purpose

The aim of this project is to develop a sign language recognition system that can identify static hand gestures using flex sensors and display the corresponding letters on an LCD screen. Real life purpose of this system is to assist communication for individuals who use sign language by translating physical gestures into readable characters in real-time.

Design Specification

The design recognizes 26 letter and a control (space) gestures based on 5 finger inputs:

- Thumb is detected using a button: Open or closed (o, c)
- State of fingers is measured using 4 flex sensors connected via XADC and voltage divider: Bent, half-bent or straight (b, h, s)

	Letter	Thumb	Index	Middle	Ring	Little
1	A	o	b	b	b	b
2	B	c	s	s	s	s
3	C	o	h	b	b	b
4	D	o	s	h	h	h
5	E	c	h	h	h	h
6	F	o	h	s	s	s
7	G	o	h	h	b	b
8	H	c	b	s	h	s
9	I	c	b	b	b	s
10	J	c	b	b	b	h
11	K	o	s	s	b	b
12	L	o	s	b	b	b
13	M	c	h	h	h	b
14	N	c	h	h	b	b
15	O	o	h	h	h	h
16	P	o	s	h	b	b
17	Q	o	h	h	h	s
18	R	o	h	s	b	b
19	S	c	b	h	h	s
20	T	c	s	s	s	b
21	U	o	s	b	s	b
22	V	c	s	s	b	b
23	W	c	s	h	s	b
24	X	c	h	b	b	b
25	Y	o	b	b	b	s
26	Z	c	s	s	b	s
31	lettertoletter	o	s	s	s	s
30	space	c	b	b	b	b

Table 1: Letters and their equivalent gestures inspired by ASL

Inputs:

- 4 analog flex sensors (to XADC via a voltage divider)
- 1 thumb button
- 2 threshold setting buttons (upper_btn and lower_btn)
- 4-bit switch to select input/output modes
- Cursor movement buttons (cursor_left_btn, cursor_right_btn)
- Reset switch for LCD displayer (reset_LCD)
- Reset button for thresholds and system (reset)

Outputs:

- LCD to display detected characters
- LEDs to display sensor values or finger states
- Seven-segment display to show the recognized letter code

Methodology

The system begins with ADC integration using the XADC module and voltage divider, which samples analog voltages from the flex sensors and converts them into 12-bit digital values. These digital values are then classified into three discrete finger states through thresholds. Threshold values for each finger are set manually using the designated buttons.

Once the classification is complete, the resulting finger states are concatenated with the thumb button state to create a 13-bit gesture code. This code is then mapped to a corresponding 5-bit letter code through a decoding module. The letter code is subsequently converted into an 8-bit ASCII value using a converter module.

This ASCII character is sent to an FSM-based LCD driver that manages character positioning and writing using appropriate timing signals. Cursor movement on the LCD is handled manually by the user through additional buttons. A character is only written to the LCD after being continuously recognized for a specific duration.

Results

The final system demonstrated successful real-time recognition and display of hand gestures representing the English alphabet. Each gesture produced consistent ADC readings which resulted in accurate letter detection. The LCD module was able to display characters at the correct positions, and cursor control functioned as intended. Additionally, the LED and seven-segment display outputs facilitated easy debugging and provided real-time feedback on system performance during testing. Throughout the experiment, no critical errors were encountered in the hardware integration or FSM transitions, and all test cases for character recognition passed under various hand positioning conditions. Calibration with adjustable thresholds for each finger significantly improved classification precision.

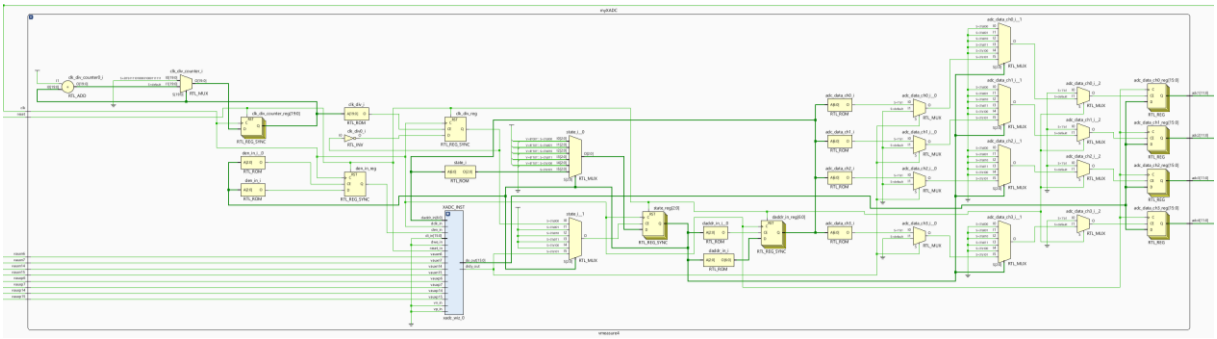

Figure 1: RTL schematic of XADC value reading module (vmeasure4)

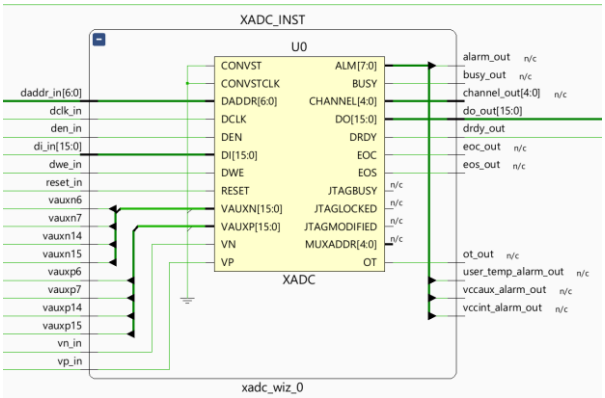

Figure 2: XADC IP insanitation

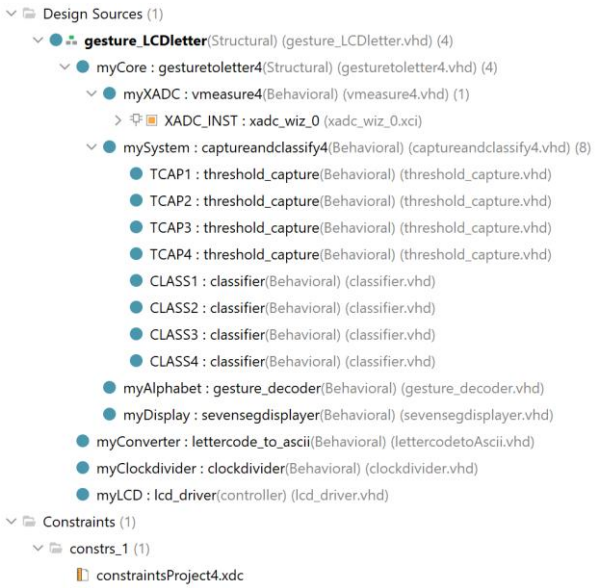

Figure 3: All modules

The project not only demonstrated isolated letter recognition but also validated continuous word construction. Physical setup, where the Basys3 board is connected to flex sensors and drives the LCD screen that outputs recognized letters can be seen at Figure 4-5 with an example displayed word "BILKENT EE" (Figure 6).

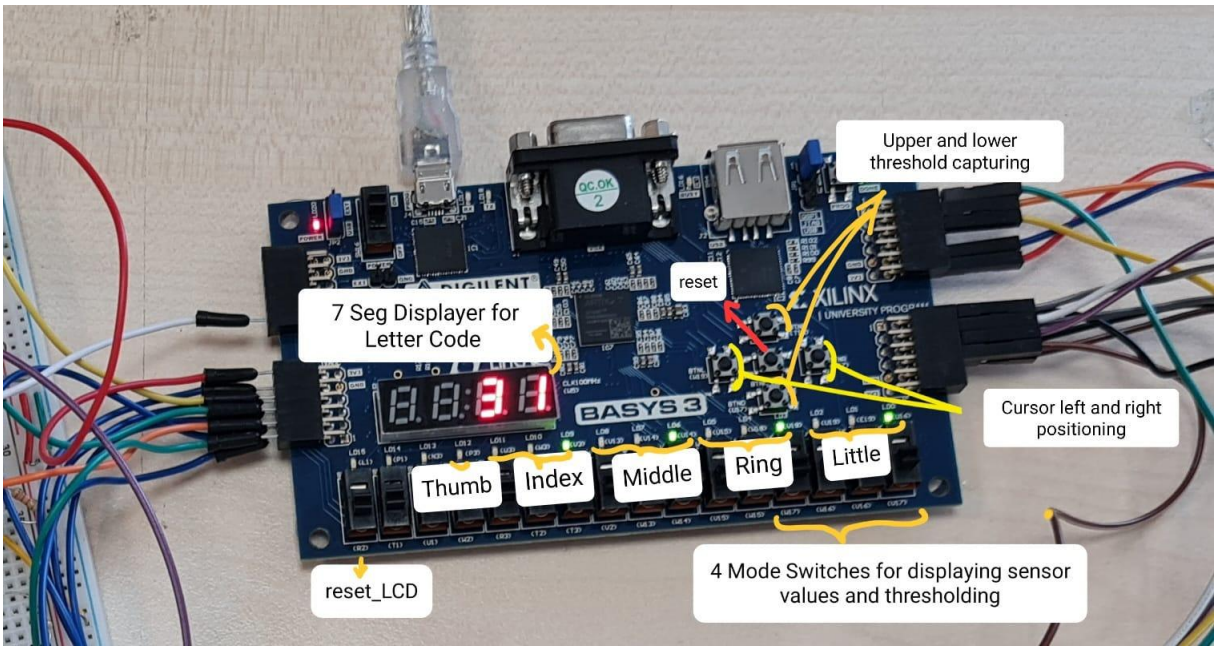

Figure 4: All components used on BASYS3

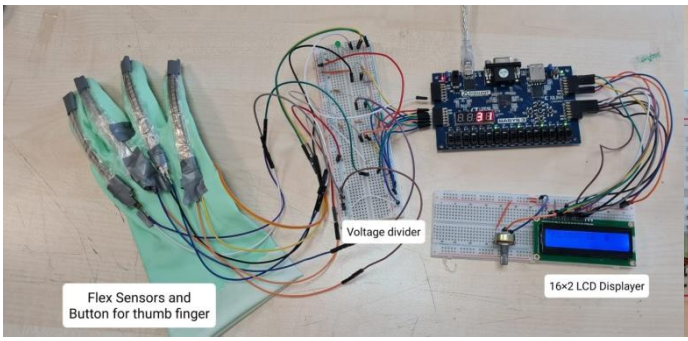

Figure 5: Physical setup

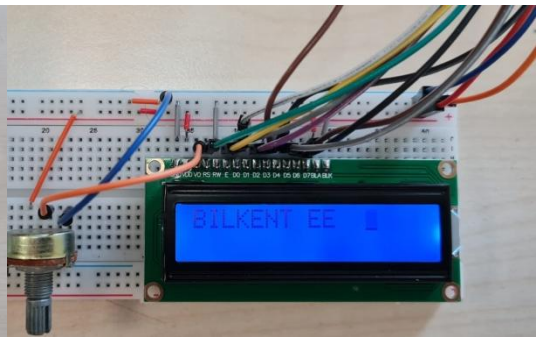

Figure 6: Displayed word

Conclusion

This project successfully achieved its purpose by implementing a real-time gesture recognition system using BASYS3 board. Each component, from ADC integration to LCD output, was tested and validated. Each gesture produced stable ADC readings that were accurately classified and mapped to the correct letters. These letters were then displayed in the correct order on the LCD screen with precise cursor positioning. The character stabilization mechanism ensured that only deliberate gestures were recorded.

The system is modular, allowing for the addition of new gestures or refinement of classification algorithms. The project reinforces digital design principles and FSM-based control methods. Overall, the system enabled the user to write complete words and insert spaces using only hand gestures without any character separation logic but with only a stability-based writing mechanism.

Appendices

Video link: https://youtu.be/R-E0OM75p7w

VHDL Codes:

1) **gesture_LCDletter**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gesture_LCDletter is
   Port (
      clk        : in  std_logic;
      reset      : in  std_logic;
      reset_LCD   : in  std_logic;

      cursor_left_btn  : in std_logic;
      cursor_right_btn : in std_logic;

      upper_btn   : in  std_logic;
      lower_btn   : in  std_logic;
      thumb_btn   : in  std_logic;
      sw_in       : in  std_logic_vector(3 downto 0);
      vauxp6, vauxn6, vauxp7, vauxn7 : in std_logic;
      vauxp14, vauxn14, vauxp15, vauxn15 : in std_logic;

      leds       : out std_logic_vector(12 downto 0);
      anodes      : out std_logic_vector(3 downto 0);
      cathodes    : out std_logic_vector(6 downto 0);

      LCD_RS      : out STD_LOGIC;
      LCD_RW      : out STD_LOGIC;
      LCD_EN      : out STD_LOGIC;
      LCD_DATA    : out STD_LOGIC_VECTOR(7 downto 0)
   );
end gesture_LCDletter;

architecture Structural of gesture_LCDletter is

   signal letter_code  : std_logic_vector(4 downto 0);
   signal input_letter : std_logic_vector(7 downto 0);

   signal letter_ascii : std_logic_vector(7 downto 0) := x"41";
   signal previous_letter : std_logic_vector(7 downto 0) := x"00";
   signal pos : std_logic_vector(3 downto 0) := "0000";
   signal clock_1khz   : std_logic ;
   signal counter : integer := 0;
   signal thumb: std_logic := not thumb_btn ;
   signal wr_char        : std_logic := '0';

   signal wr_pending     : std_logic := '0';
   signal latched_char   : std_logic_vector(7 downto 0) := (others => '0');
   signal latched_pos    : std_logic_vector(3 downto 0) := (others => '0');

   signal last_left       : std_logic := '0';
   signal last_right      : std_logic := '0';

component clockdivider is Port (
      internalclk : in STD_LOGIC;
      speedselect : in STD_LOGIC_VECTOR (1 downto 0);
      obtainedclk : out STD_LOGIC); end component;

component LCD_Driver
   Port (
      clk       : IN    STD_LOGIC;
      rst       : IN    STD_LOGIC;
      rw, rs, e  : OUT   STD_LOGIC;
      lcd_data   : OUT   STD_LOGIC_VECTOR(7 DOWNTO 0);
      char       : IN    STD_LOGIC_VECTOR(7 DOWNTO 0);
      position   : IN    STD_LOGIC_VECTOR(3 DOWNTO 0);
      wr_char    : IN    STD_LOGIC
   );
end component;

   component gesturetoletter4
      Port (
         clk, reset       : in  std_logic;
```

```
        upper_btn, lower_btn : in  std_logic;
        vauxp6, vauxn6, vauxp7, vauxn7 : in std_logic;
        vauxp14, vauxn14, vauxp15, vauxn15 : in std_logic;
        thumb_btn         : in std_logic;
        sw_in             : in std_logic_vector(3 downto 0);
        leds              : out std_logic_vector(12 downto 0);
        letter_code       : out std_logic_vector(4 downto 0);
        anodes            : out std_logic_vector(3 downto 0);
        cathodes          : out std_logic_vector(6 downto 0)
    );
  end component;

  component lettercode_to_ascii
    Port (
        letter_code  : in  std_logic_vector(4 downto 0);
        letter_ascii : out std_logic_vector(7 downto 0));
  end component;

begin

  myCore: gesturetoletter4
    port map (
        clk => clk, reset => reset,
        upper_btn => upper_btn,
        lower_btn => lower_btn,
        thumb_btn => thumb,
        sw_in => sw_in,
        vauxp6 => vauxp6, vauxn6 => vauxn6,
        vauxp7 => vauxp7, vauxn7 => vauxn7,
        vauxp14 => vauxp14, vauxn14 => vauxn14,
        vauxp15 => vauxp15, vauxn15 => vauxn15,
        leds => leds,
        letter_code => letter_code,
        anodes => anodes,
        cathodes => cathodes);

  myConverter: lettercode_to_ascii
    port map (
        letter_code => letter_code,
        letter_ascii => input_letter);

  process(clock_1kHz, reset_LCD)
begin
  if reset_LCD = '1' then
    last_left  <= '0';
    last_right <= '0';

    counter <= 0;
    previous_letter <= (others => '0');
    pos <= "0000";
    letter_ascii <= (others => '0');
    wr_char <= '0';
    wr_pending <= '0';
  elsif rising_edge(clock_1kHz) then
    wr_char <= '0';

    if input_letter = previous_letter and input_letter /= x"00" then
        if counter < 2000 then
            counter <= counter + 1;
        else
            latched_char <= input_letter;
            latched_pos <= pos;
            pos <= std_logic_vector(unsigned(pos) + 1);
            wr_pending <= '1';
            counter <= 0;
        end if;
    else
        counter <= 0;
        previous_letter <= input_letter;
    end if;
```

```
      -- Pulse wr_char for one cycle
      if wr_pending = '1' then
         wr_char <= '1';
         wr_pending <= '0';
      else
         if cursor_right_btn = '1' and last_right = '0' then
            if unsigned(pos) < 15 then
               pos <= std_logic_vector(unsigned(pos) + 1);
            end if;
         end if;
         if cursor_left_btn = '1' and last_left = '0' then
            if unsigned(pos) > 0 then
               pos <= std_logic_vector(unsigned(pos) - 1);
            end if;
         end if;
      end if;

      last_left  <= cursor_left_btn;
      last_right <= cursor_right_btn;

   end if;
end process;

   myClockdivider: clockdivider port map(
      internalclk => clk,
      speedselect => "00",
      obtainedclk => clock_1khz);

   myLCD: LCD_Driver port map (
         CLK      => clk,
         RST      => reset_LCD,
         CHAR     => latched_char,
         position => latched_pos,
         wr_char  => wr_char,
         RS       => LCD_RS,
         RW       => LCD_RW,
         E        => LCD_EN,
         LCD_DATA => LCD_DATA);
end Structural;
```

2) lettercode_to_ascii

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity lettercode_to_ascii is
   Port (
      letter_code   : in  STD_LOGIC_VECTOR (4 downto 0);
      letter_ascii  : out STD_LOGIC_VECTOR (7 downto 0)
   );
end lettercode_to_ascii;

architecture Behavioral of lettercode_to_ascii is
   signal code_int : integer := to_integer(unsigned(letter_code));
begin

   process(letter_code)
   begin
      case letter_code is
         when "11110" =>
            letter_ascii <= x"20"; -- Space
         when "11111" =>
            letter_ascii <= x"00"; -- for letter-to-letter
         when "00000" =>
            letter_ascii <= x"00"; -- for default/undefined
         when others =>
            letter_ascii <= std_logic_vector(to_unsigned(64 + code_int, 8)); -- ASCII A-Z
      end case;
   end process;
end Behavioral;
```

3) clockdivider

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity clockdivider is
   Port ( internalclk : in STD_LOGIC;
          speedselect : in STD_LOGIC_VECTOR (1 downto 0);
          obtainedclk : out STD_LOGIC);
end clockdivider;

architecture Behavioral of clockdivider is

signal count : integer range 0 to 100_000_000 := 0;
signal divisionfactor : integer := 100000000;
signal clockwaveform : std_logic := '0';

begin

process(internalclk) begin
   if rising_edge(internalclk) then
      case speedselect is
         when "00" => divisionfactor <= 100000;  -- 1 kHz
         when "01" => divisionfactor <= 100;  -- 1 MHz
         when "10" => divisionfactor <= 1000000;  -- 100 Hz
         when "11" => divisionfactor <= 100000000; -- 1 Hz
         when others => divisionfactor <= 100000000; -- Default case
end case; end if; end process;

process(internalclk) begin
if rising_edge(internalclk) then
   if count = divisionfactor / 2 then
   clockwaveform <= not clockwaveform; count <= 0;
   else count <= count + 1 ;
   end if;
end if; end process;

obtainedclk <= clockwaveform;

end Behavioral;
```

4) lcd_driver

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY lcd_driver IS
  PORT(
    clk       : IN    STD_LOGIC;
    rst       : IN    STD_LOGIC;
    rw, rs, e  : OUT   STD_LOGIC;
    lcd_data   : OUT   STD_LOGIC_VECTOR(7 DOWNTO 0);
    char       : IN    STD_LOGIC_VECTOR(7 DOWNTO 0);
    position   : IN    STD_LOGIC_VECTOR(3 DOWNTO 0);
    wr_char    : IN    STD_LOGIC
  );
END lcd_driver;

ARCHITECTURE controller OF lcd_driver IS
  TYPE CONTROL IS(power_up, initialize, set_position, write_char, idle);
  SIGNAL state         : CONTROL;
  CONSTANT freq        : INTEGER := 100; --system clock freq in MHz
  SIGNAL pos_addr      : STD_LOGIC_VECTOR(7 DOWNTO 0);
BEGIN

  PROCESS(clk)
    VARIABLE clk_count : INTEGER := 0; --event counter for timing
  BEGIN
    if rising_edge(clk) then

      CASE state IS
```

```
--wait 50 ms to ensure Vdd has risen and required LCD wait is met
WHEN power_up =>
  IF(clk_count < (50000 * freq)) THEN    --wait 50 ms
    clk_count := clk_count + 1;
    state <= power_up;
  ELSE
    clk_count := 0;
    rs <= '0';
    rw <= '0';
    lcd_data <= "00110000";
    state <= initialize;
  END IF;

--cycle through initialization sequence
WHEN initialize =>
  clk_count := clk_count + 1;
  IF(clk_count < (10 * freq)) THEN       --function set
    lcd_data <= "00111100";      --2-line mode, display on
    e <= '1';
    state <= initialize;
  ELSIF(clk_count < (60 * freq)) THEN    --wait 50 us
    lcd_data <= "00000000";
    e <= '0';
    state <= initialize;
  ELSIF(clk_count < (70 * freq)) THEN    --display on/off control
    lcd_data <= "00001111";      --display on, cursor on, blink on
    e <= '1';
    state <= initialize;
  ELSIF(clk_count < (120 * freq)) THEN   --wait 50 us
    lcd_data <= "00000000";
    e <= '0';
    state <= initialize;
  ELSIF(clk_count < (130 * freq)) THEN   --display clear
    lcd_data <= "00000001";
    e <= '1';
    state <= initialize;
  ELSIF(clk_count < (2130 * freq)) THEN  --wait 2 ms
    lcd_data <= "00000000";
    e <= '0';
    state <= initialize;
  ELSIF(clk_count < (2140 * freq)) THEN  --entry mode set
    lcd_data <= "00000110";      --increment mode, entire shift off
    e <= '1';
    state <= initialize;
  ELSIF(clk_count < (2200 * freq)) THEN  --wait 60 us
    lcd_data <= "00000000";
    e <= '0';
    state <= initialize;
  ELSE                                   --initialization complete
    clk_count := 0;
    state <= idle;
  END IF;

WHEN idle =>
  IF wr_char = '1' THEN
    pos_addr <= "10000000" OR ("0000" & position);
    clk_count := 0;
    state <= set_position;
  ELSE
    state <= idle;
  END IF;

WHEN set_position =>
  IF(clk_count < (50 * freq)) THEN      --do not exit for 50us
    IF(clk_count < freq) THEN           --negative enable
      lcd_data <= pos_addr;             -- Position command
      rs <= '0';                        -- Command mode
      rw <= '0';                        -- Write mode
      e <= '0';
    ELSIF(clk_count < (14 * freq)) THEN  --positive enable half-cycle
```

```
            e <= '1';
          ELSIF(clk_count < (27 * freq)) THEN  --negative enable half-cycle
            e <= '0';
          END IF;
          clk_count := clk_count + 1;
          state <= set_position;
        ELSE
          clk_count := 0;
          state <= write_char;
        END IF;

      WHEN write_char =>
        IF(clk_count < (50 * freq)) THEN      --do not exit for 50us
          IF(clk_count < freq) THEN          --negative enable
            lcd_data <= char;                -- Character data
            rs <= '1';                       -- Data mode
            rw <= '0';                       -- Write mode
            e <= '0';
          ELSIF(clk_count < (14 * freq)) THEN  --positive enable half-cycle
            e <= '1';
          ELSIF(clk_count < (27 * freq)) THEN  --negative enable half-cycle
            e <= '0';
          END IF;
          clk_count := clk_count + 1;
          state <= write_char;
        ELSE
          clk_count := 0;
          state <= idle;
        END IF;
    END CASE;

IF (rst = '1') THEN
   state <= power_up;
   clk_count := 0;
   pos_addr <= (others => '0');
END IF;

   END IF;
  END PROCESS;
END controller;
```

5) gesturetoletter4

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gesturetoletter4 is
   Port (
      clk        : in  std_logic;
      reset      : in  std_logic;
      upper_btn  : in  std_logic;
      lower_btn  : in  std_logic;

      vauxp6     : in  std_logic;
      vauxn6     : in  std_logic;
      vauxp7     : in  std_logic;
      vauxn7     : in  std_logic;
      vauxp14    : in  std_logic;
      vauxn14    : in  std_logic;
      vauxp15    : in  std_logic;
      vauxn15    : in  std_logic;

      thumb_btn  : in  std_logic;
      sw_in      : in  std_logic_vector(3 downto 0);

      leds       : out std_logic_vector(12 downto 0);
      letter_code : out std_logic_vector(4 downto 0);
      anodes     : out std_logic_vector(3 downto 0);
      cathodes   : out std_logic_vector(6 downto 0) );
end gesturetoletter4;
```

```
architecture Structural of gesturetoletter4 is

   signal adc1, adc2, adc3, adc4 : std_logic_vector(11 downto 0);
   signal f1, f2, f3, f4        : std_logic_vector(2 downto 0);
   signal thmode               : std_logic_vector(1 downto 0);
   signal gesture_code          : std_logic_vector(12 downto 0);
   signal letter_internal         : std_logic_vector(4 downto 0);

   component vmeasure4
      Port (
         clk, reset     : in std_logic;
         vauxp6, vauxn6, vauxp7, vauxn7, vauxp14, vauxn14, vauxp15, vauxn15 : in std_logic;
         adc1, adc2, adc3, adc4 : out std_logic_vector(11 downto 0)
      );
   end component;

   component captureandclassify4
      Port (
         clk, reset : in std_logic;
         thmode     : in std_logic_vector(1 downto 0);
         enable1    : in std_logic;
         enable2    : in std_logic;
         enable3    : in std_logic;
         enable4    : in std_logic;
         adc1, adc2, adc3, adc4 : in std_logic_vector(11 downto 0);
         finger1_state, finger2_state, finger3_state, finger4_state : out std_logic_vector(2 downto 0)
      );
   end component;

   component gesture_decoder
      Port (
         gesture_code : in std_logic_vector(12 downto 0);
         letter_code  : out std_logic_vector(4 downto 0)
      );
   end component;

   component sevensegdisplayer
      Port (
      clk      : in  STD_LOGIC;
      value_in : in  STD_LOGIC_VECTOR (4 downto 0);
      an       : out STD_LOGIC_VECTOR (3 downto 0);
      seg_out  : out STD_LOGIC_VECTOR (6 downto 0)
      );
   end component;

begin

   thmode(1) <= upper_btn;
   thmode(0) <= lower_btn;

process(sw_in, adc1, adc2, adc3, adc4, gesture_code) begin
   if sw_in = "0001" then
      leds <= '0' & adc1;
   elsif sw_in = "0010" then
      leds <= '0' & adc2 ;
   elsif sw_in = "0100" then
      leds <= '0' & adc3;
   elsif sw_in = "1000" then
      leds <= '0' & adc4;
   else
      leds <= gesture_code;
   end if;
end process;

   myXADC: vmeasure4
      port map (
         clk => clk, reset => reset,
         vauxp6 => vauxp6, vauxn6 => vauxn6,
         vauxp7 => vauxp7, vauxn7 => vauxn7,
         vauxp14 => vauxp14, vauxn14 => vauxn14,
         vauxp15 => vauxp15, vauxn15 => vauxn15,
```

```
        adc1 => adc1, adc2 => adc2, adc3 => adc3, adc4 => adc4  );

    mySystem: captureandclassify4
      port map (
        clk => clk,
        reset => reset,
        thmode => thmode,
        enable1 => sw_in(0), enable2 => sw_in(1), enable3 => sw_in(2), enable4 => sw_in(3),
        adc1 => adc1, adc2 => adc2, adc3 => adc3, adc4 => adc4,
        finger1_state => f1, finger2_state => f2, finger3_state => f3, finger4_state => f4 );

    gesture_code <= thumb_btn & f1 & f2 & f3 & f4;
    letter_code <= letter_internal;

    myAlphabet: gesture_decoder
      port map (
        gesture_code => gesture_code,
        letter_code  => letter_internal);

     myDisplay: sevensegdisplayer
      port map (
        clk      => clk,
        value_in => letter_internal,
        an       => anodes,
        seg_out  => cathodes);
end Structural;
```

6) captureandclassify4

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity captureandclassify4 is
    Port (
      clk        : in  std_logic;
      reset      : in  std_logic;
      thmode     : in  std_logic_vector(1 downto 0);
      enable1    : in  std_logic;
      enable2    : in  std_logic;
      enable3    : in  std_logic;
      enable4    : in  std_logic;
      adc1       : in  std_logic_vector(11 downto 0);
      adc2       : in  std_logic_vector(11 downto 0);
      adc3       : in  std_logic_vector(11 downto 0);
      adc4       : in  std_logic_vector(11 downto 0);
      finger1_state : out std_logic_vector(2 downto 0);
      finger2_state : out std_logic_vector(2 downto 0);
      finger3_state : out std_logic_vector(2 downto 0);
      finger4_state : out std_logic_vector(2 downto 0)
    );
end captureandclassify4;

architecture Behavioral of captureandclassify4 is

    signal t1_f1, t2_f1 : std_logic_vector(11 downto 0);
    signal t1_f2, t2_f2 : std_logic_vector(11 downto 0);
    signal t1_f3, t2_f3 : std_logic_vector(11 downto 0);
    signal t1_f4, t2_f4 : std_logic_vector(11 downto 0);

    signal threshold_mode, classify: std_logic;
    signal capture1, capture2, capture3, capture4 : std_logic;

    component threshold_capture
      Port (
        clk          : in  std_logic;
        reset        : in  std_logic;
        enable       : in  std_logic;
        adc_value    : in  std_logic_vector(11 downto 0);
        threshold_mode : in  std_logic;
        threshold1   : out std_logic_vector(11 downto 0);
        threshold2   : out std_logic_vector(11 downto 0)
```

```
    );
  end component;

  component classifier
    Port (
      clk         : in  std_logic;
      reset       : in  std_logic;
      enable      : in  std_logic;
      adc_value   : in  std_logic_vector(11 downto 0);
      threshold1  : in  std_logic_vector(11 downto 0);
      threshold2  : in  std_logic_vector(11 downto 0);
      finger_state : out std_logic_vector(2 downto 0)
    );
  end component;

begin

  capture1 <= enable1 and (thmode(1) xor thmode(0));
  capture2 <= enable2 and (thmode(1) xor thmode(0));
  capture3 <= enable3 and (thmode(1) xor thmode(0));
  capture4 <= enable4 and (thmode(1) xor thmode(0));
  threshold_mode <= thmode(1) and not thmode(0);
  classify <= not (enable1 or enable2 or enable3 or enable4);

  TCAP1: threshold_capture
    port map (
      clk => clk, reset => reset, enable => capture1,
      adc_value => adc1, threshold_mode => threshold_mode,
      threshold1 => t1_f1, threshold2 => t2_f1
    );

  TCAP2: threshold_capture
    port map (
      clk => clk, reset => reset, enable => capture2,
      adc_value => adc2, threshold_mode => threshold_mode,
      threshold1 => t1_f2, threshold2 => t2_f2
    );

  TCAP3: threshold_capture
    port map (
      clk => clk, reset => reset, enable => capture3,
      adc_value => adc3, threshold_mode => threshold_mode,
      threshold1 => t1_f3, threshold2 => t2_f3
    );

  TCAP4: threshold_capture
    port map (
      clk => clk, reset => reset, enable => capture4,
      adc_value => adc4, threshold_mode => threshold_mode,
      threshold1 => t1_f4, threshold2 => t2_f4
    );

  -- Classify fingers
  CLASS1: classifier
    port map (
      clk => clk, reset => reset, enable => classify,
      adc_value => adc1, threshold1 => t1_f1, threshold2 => t2_f1,
      finger_state => finger1_state
    );

  CLASS2: classifier
    port map (
      clk => clk, reset => reset, enable => classify,
      adc_value => adc2, threshold1 => t1_f2, threshold2 => t2_f2,
      finger_state => finger2_state
    );

  CLASS3: classifier
    port map (
      clk => clk, reset => reset, enable => classify,
      adc_value => adc3, threshold1 => t1_f3, threshold2 => t2_f3,
```

```
        finger_state => finger3_state
    );

  CLASS4: classifier
    port map (
        clk => clk, reset => reset, enable => classify,
        adc_value => adc4, threshold1 => t1_f4, threshold2 => t2_f4,
        finger_state => finger4_state
    );
end Behavioral;
```

7) threshold_capture

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity threshold_capture is
  Port (
    clk           : in  std_logic;
    reset         : in  std_logic;
    enable        : in  std_logic;
    adc_value     : in  std_logic_vector(11 downto 0);
    threshold_mode : in  std_logic;          -- '0' to write T1, '1' to write T2
    threshold1    : out std_logic_vector(11 downto 0);
    threshold2    : out std_logic_vector(11 downto 0));
end threshold_capture;


architecture Behavioral of threshold_capture is

signal t1_def : std_logic_vector(11 downto 0) := "110110100001"; -- 3489
signal t2_def : std_logic_vector(11 downto 0) := "110111100000"; -- 3560

  signal t1_reg : std_logic_vector(11 downto 0) := t1_def;
  signal t2_reg : std_logic_vector(11 downto 0) := t2_def;

begin

  process(clk) begin
    if rising_edge(clk) then
      if reset = '1' then
        t1_reg <= t1_def;
        t2_reg <= t2_def;
      elsif enable = '1' then
        if threshold_mode = '0' then
          t1_reg <= adc_value;
        else
          t2_reg <= adc_value;
    end if; end if; end if;
  end process;

  threshold1 <= t1_reg;
  threshold2 <= t2_reg;

end Behavioral;
```

8) classifier

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity classifier is
  Port (
    clk         : in  std_logic;
    reset       : in  std_logic;
    enable      : in  std_logic;
    adc_value   : in  std_logic_vector(11 downto 0);
    threshold1  : in  std_logic_vector(11 downto 0);
    threshold2  : in  std_logic_vector(11 downto 0);
    finger_state : out std_logic_vector(2 downto 0));
end classifier;
```

```
architecture Behavioral of classifier is

signal state_reg : std_logic_vector(2 downto 0) := "000";

begin
   process(clk) begin if rising_edge(clk) then
         if reset = '1' then
            state_reg <= "000";  -- straight by default
         elsif enable = '1' then
            if adc_value < threshold1 then
               state_reg <= "001";  -- straight
            elsif adc_value < threshold2 then
               state_reg <= "010";  -- half bent
            else
               state_reg <= "100";  -- fully bent
         end if; end if; end if;
   end process;
finger_state <= state_reg;

end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity classifier is
   Port (
      clk          : in  std_logic;
      reset        : in  std_logic;
      enable       : in  std_logic;
      adc_value    : in  std_logic_vector(11 downto 0);
      threshold1   : in  std_logic_vector(11 downto 0);
      threshold2   : in  std_logic_vector(11 downto 0);
      finger_state : out std_logic_vector(2 downto 0));
end classifier;

architecture Behavioral of classifier is

signal state_reg : std_logic_vector(2 downto 0) := "000";

begin
   process(clk) begin if rising_edge(clk) then
         if reset = '1' then
            state_reg <= "000";  -- straight by default
         elsif enable = '1' then
            if adc_value < threshold1 then
               state_reg <= "001";  -- straight
            elsif adc_value < threshold2 then
               state_reg <= "010";  -- half bent
            else
               state_reg <= "100";  -- fully bent
         end if; end if; end if;
   end process;
finger_state <= state_reg;

end Behavioral;
```

9) vmeasure4

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity vmeasure4 is
   Port (
      clk        : in  std_logic;
      reset      : in STD_LOGIC;
      vauxp6     : in  std_logic;
      vauxn6     : in  std_logic;
      vauxp7     : in  std_logic;
      vauxn7     : in  std_logic;
```

```
        vauxp14    : in  std_logic;
        vauxn14    : in  std_logic;
        vauxp15    : in  std_logic;
        vauxn15    : in  std_logic;
        adc1       : out std_logic_vector(11 downto 0);
        adc2       : out std_logic_vector(11 downto 0);
        adc3       : out std_logic_vector(11 downto 0);
        adc4       : out std_logic_vector(11 downto 0)
    );
end vmeasure4;

architecture Behavioral of vmeasure4 is

    -- XADC component declaration
    component xadc_wiz_0 is
        port (
            daddr_in        : in  STD_LOGIC_VECTOR (6 downto 0);
            den_in          : in  STD_LOGIC;
            di_in           : in  STD_LOGIC_VECTOR (15 downto 0);
            dwe_in          : in  STD_LOGIC;
            do_out          : out STD_LOGIC_VECTOR (15 downto 0);
            drdy_out        : out STD_LOGIC;
            dclk_in         : in  STD_LOGIC;
            reset_in        : in  STD_LOGIC;
            vauxp6          : in  STD_LOGIC;
            vauxn6          : in  STD_LOGIC;
            vauxp7          : in  STD_LOGIC;
            vauxn7          : in  STD_LOGIC;
            vauxp14         : in  STD_LOGIC;
            vauxn14         : in  STD_LOGIC;
            vauxp15         : in  STD_LOGIC;
            vauxn15         : in  STD_LOGIC;
            busy_out        : out STD_LOGIC;
            channel_out     : out STD_LOGIC_VECTOR (4 downto 0);
            eoc_out         : out STD_LOGIC;
            eos_out         : out STD_LOGIC;
            alarm_out       : out STD_LOGIC;
            vp_in           : in  STD_LOGIC;
            vn_in           : in  STD_LOGIC
        );
    end component;

    -- Constants for XADC channel addresses
    constant ADDR_CHAN0  : std_logic_vector(6 downto 0) := "0010110"; -- Channel 6 (VAUXP/N6)
    constant ADDR_CHAN1  : std_logic_vector(6 downto 0) := "0010111"; -- Channel 7 (VAUXP/N7)
    constant ADDR_CHAN2  : std_logic_vector(6 downto 0) := "0011110"; -- Channel 14 (VAUXP/N14)
    constant ADDR_CHAN3  : std_logic_vector(6 downto 0) := "0011111"; -- Channel 15 (VAUXP/N15)

    -- Signals for XADC interface
    signal daddr_in      : STD_LOGIC_VECTOR (6 downto 0);
    signal den_in        : STD_LOGIC := '0';
    signal di_in         : STD_LOGIC_VECTOR (15 downto 0) := (others => '0');
    signal dwe_in        : STD_LOGIC := '0';
    signal do_out        : STD_LOGIC_VECTOR (15 downto 0);
    signal drdy_out      : STD_LOGIC;
    signal busy_out      : STD_LOGIC;
    signal channel_out   : STD_LOGIC_VECTOR (4 downto 0);
    signal eoc_out       : STD_LOGIC;
    signal eos_out       : STD_LOGIC;

    -- ADC data storage
    signal adc_data_ch0  : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal adc_data_ch1  : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal adc_data_ch2  : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal adc_data_ch3  : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');

    -- State machine for reading ADC channels
    type state_type is (IDLE, READ_CH0, READ_CH1, READ_CH2, READ_CH3, WAIT_DRDY);
    signal state : state_type := IDLE;

    -- Clock divider to slow down the state machine
```

```
    signal clk_div_counter : integer range 0 to 999999 := 0;
    signal clk_div : std_logic := '0';

begin
    -- Instantiate the XADC wizard
    XADC_INST : xadc_wiz_0
        port map (
            daddr_in => daddr_in,
            den_in => den_in,
            di_in => di_in,
            dwe_in => dwe_in,
            do_out => do_out,
            drdy_out => drdy_out,
            dclk_in => clk,
            reset_in => reset,
            vauxp6 => vauxp6,
            vauxn6 => vauxn6,
            vauxp7 => vauxp7,
            vauxn7 => vauxn7,
            vauxp14 => vauxp14,
            vauxn14 => vauxn14,
            vauxp15 => vauxp15,
            vauxn15 => vauxn15,
            busy_out => busy_out,
            channel_out => channel_out,
            eoc_out => eoc_out,
            eos_out => eos_out,
            alarm_out => open,
            vp_in => '0',
            vn_in => '0'
        );

    -- Clock divider process to slow down state machine
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                clk_div_counter <= 0;
                clk_div <= '0';
            else
                if clk_div_counter = 999999 then -- Approx 10ms at 100MHz
                    clk_div_counter <= 0;
                    clk_div <= not clk_div;
                else
                    clk_div_counter <= clk_div_counter + 1;
                end if;
            end if;
        end if;
    end process;

    -- XADC reading state machine
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                state <= IDLE;
                den_in <= '0';
                daddr_in <= (others => '0');
            else
                case state is
                    when IDLE =>
                        if clk_div = '1' then
                            state <= READ_CH0;
                        end if;

                    when READ_CH0 =>
                        daddr_in <= ADDR_CHAN0;
                        den_in <= '1';
                        state <= WAIT_DRDY;

                    when READ_CH1 =>
```

```
                daddr_in <= ADDR_CHAN1;
                den_in <= '1';
                state <= WAIT_DRDY;

            when READ_CH2 =>
                daddr_in <= ADDR_CHAN2;
                den_in <= '1';
                state <= WAIT_DRDY;

            when READ_CH3 =>
                daddr_in <= ADDR_CHAN3;
                den_in <= '1';
                state <= WAIT_DRDY;

            when WAIT_DRDY =>
                den_in <= '0';

                if drdy_out = '1' then
                    -- Store the data based on current address
                    case daddr_in is
                        when ADDR_CHAN0 =>
                            adc_data_ch0 <= do_out;
                            state <= READ_CH1;
                        when ADDR_CHAN1 =>
                            adc_data_ch1 <= do_out;
                            state <= READ_CH2;
                        when ADDR_CHAN2 =>
                            adc_data_ch2 <= do_out;
                            state <= READ_CH3;
                        when ADDR_CHAN3 =>
                            adc_data_ch3 <= do_out;
                            state <= IDLE;
                        when others =>
                            state <= IDLE;
                    end case;
                end if;
            end case;
        end if;
    end if;
  end process;

adc1 <= adc_data_ch0(15 downto 4);
adc2 <= adc_data_ch1(15 downto 4);
adc3 <= adc_data_ch2(15 downto 4);
adc4 <= adc_data_ch3(15 downto 4);

end Behavioral;
```

10) gesture_decoder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity gesture_decoder is
  Port (
    gesture_code : in  std_logic_vector(12 downto 0);
    letter_code  : out std_logic_vector(4 downto 0)
  );
end gesture_decoder;

architecture Behavioral of gesture_decoder is
begin
  process(gesture_code)
  begin
    case gesture_code is
      when "0100100100100" => letter_code <= "00001";  -- A
      when "1001001001001" => letter_code <= "00010";  -- B
      when "0010100100100" => letter_code <= "00011";  -- C
      when "0001010010010" => letter_code <= "00100";  -- D
      when "1010010010010" => letter_code <= "00101";  -- E
      when "0010001001001" => letter_code <= "00110";  -- F
```

```
        when "0010010100100" => letter_code <= "00111";  -- G
        when "1100001010001" => letter_code <= "01000";  -- H
        when "1100100100001" => letter_code <= "01001";  -- I
        when "1100100100010" => letter_code <= "01010";  -- J
        when "0001001100100" => letter_code <= "01011";  -- K
        when "0001100100100" => letter_code <= "01100";  -- L
        when "1010010010100" => letter_code <= "01101";  -- M
        when "1010010100100" => letter_code <= "01110";  -- N
        when "0010010010010" => letter_code <= "01111";  -- O
        when "0001010100100" => letter_code <= "10000";  -- P
        when "0010010010001" => letter_code <= "10001";  -- Q
        when "0010001100100" => letter_code <= "10010";  -- R
        when "1100010010001" => letter_code <= "10011";  -- S
        when "1001001001100" => letter_code <= "10100";  -- T
        when "0001100001100" => letter_code <= "10101";  -- U
        when "1001001100100" => letter_code <= "10110";  -- V
        when "1001010001100" => letter_code <= "10111";  -- W
        when "1010100100100" => letter_code <= "11000";  -- X
        when "0100100100001" => letter_code <= "11001";  -- Y
        when "1001001100001" => letter_code <= "11010";  -- Z
        when "0001001001001" => letter_code <= "11111";  -- lettertoletter
        when "1100100100100" => letter_code <= "11110";  -- space
        when others => letter_code <= "00000";  -- default
      end case;
   end process;
end Behavioral;
```

11) sevensegdisplayer

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity sevensegdisplayer is
   Port (
      clk      : in  STD_LOGIC;                          -- 100 MHz clock
      value_in : in  STD_LOGIC_VECTOR (4 downto 0);      -- 5-bit binary input (0-31)
      an       : out STD_LOGIC_VECTOR (3 downto 0);      -- anodes (active low)
      seg_out      : out STD_LOGIC_VECTOR (6 downto 0)        -- segments a to g
   );
end sevensegdisplayer;

architecture Behavioral of sevensegdisplayer is

   signal digit_ones  : STD_LOGIC_VECTOR(3 downto 0);
   signal digit_tens  : STD_LOGIC_VECTOR(3 downto 0);
   signal mux_select  : STD_LOGIC := '0';
   signal seg     : STD_LOGIC_VECTOR(6 downto 0);
   signal clk_div     : STD_LOGIC := '0';
   signal div_counter : INTEGER range 0 to 49999 := 0;

begin

   -- Clock Divider
   process(clk)
   begin
      if rising_edge(clk) then
         if div_counter = 49999 then
            div_counter <= 0;
            clk_div <= not clk_div;
         else
            div_counter <= div_counter + 1;
         end if;
      end if;
   end process;

   process(clk_div)
   begin
      if rising_edge(clk_div) then
         mux_select <= not mux_select;
      end if;
   end process;
```

```
    process(value_in)
        variable val : INTEGER;
    begin
        val := to_integer(unsigned(value_in));
        digit_tens <= std_logic_vector(to_unsigned(val / 10, 4));
        digit_ones <= std_logic_vector(to_unsigned(val mod 10, 4));
    end process;

    process(mux_select, digit_tens, digit_ones)
    begin
        case mux_select is
            when '0' =>
                an <= "1110";
                case digit_ones is
                    when "0000" => seg <= "1000000"; -- 0
                    when "0001" => seg <= "1111001"; -- 1
                    when "0010" => seg <= "0100100"; -- 2
                    when "0011" => seg <= "0110000"; -- 3
                    when "0100" => seg <= "0011001"; -- 4
                    when "0101" => seg <= "0010010"; -- 5
                    when "0110" => seg <= "0000010"; -- 6
                    when "0111" => seg <= "1111000"; -- 7
                    when "1000" => seg <= "0000000"; -- 8
                    when "1001" => seg <= "0010000"; -- 9
                    when others => seg <= "1111111"; -- blank
                end case;
            when others =>
                an <= "1101";
                case digit_tens is
                    when "0000" => seg <= "1000000"; -- 0
                    when "0001" => seg <= "1111001"; -- 1
                    when "0010" => seg <= "0100100"; -- 2
                    when "0011" => seg <= "0110000"; -- 3
                    when others => seg <= "1111111"; -- blank
                end case;
        end case;
    end process;

    seg_out <= seg;

end Behavioral;
```